

Agentic AI Fundamentals

Q1

What is Agentic AI and how is it fundamentally different from traditional LLM applications?

Answer:

Agentic AI refers to AI systems that can **perceive, plan, decide, act, and iterate autonomously** to achieve goals — without step-by-step human instruction. Traditional LLM applications are **single-turn or pipeline-based**: you send a prompt, get a response, done. Agentic AI is **iterative and autonomous**: the agent evaluates results, decides next actions, uses tools, and continues until the goal is achieved. Key differences: (1) **Autonomy** — agent makes decisions independently. (2) **Multi-step execution** — handles tasks requiring many sequential steps. (3) **Tool use** — calls APIs, searches web, writes code, reads files. (4) **Self-correction** — if an action fails, agent retries with a different approach. (5) **Goal-oriented** — works toward an objective, not just answering a question.

■ **Example:**

Traditional LLM app:

User: 'Summarize this document'

LLM: Returns summary -> DONE (1 step, no tools, no iteration)

Agentic AI:

User: 'Research quantum computing trends and write a report'

Agent Step 1: Searches web for 'quantum computing 2026 trends'

Agent Step 2: Evaluates results -> 'I need more technical depth'

Agent Step 3: Searches arXiv for recent papers

Agent Step 4: Reads top 5 papers using document reader tool

Agent Step 5: Synthesizes findings

Agent Step 6: Writes structured report

Agent Step 7: Reviews report -> 'Missing section on applications'

Agent Step 8: Fills gap, finalizes report -> DONE (8 steps, autonomous)

Same goal, completely different architecture.

■ *Interview Tip: Say 'The key difference is autonomy + iteration. Traditional LLMs answer questions. Agents accomplish goals.' Then mention the 5 properties: perceive, plan, decide, act, iterate. This shows you understand the fundamental shift.*

Q2

What are the core components every production AI agent must have?

Answer:

Five essential components: (1) **Brain (LLM)** — the reasoning engine that decides what to do next. The LLM reads the current state and determines the next action. (2) **Tools** — the agent's hands. Functions it

can call: web search, code executor, file reader, API caller, database query. The agent decides WHEN and HOW to use them. (3) **Memory** — short-term (current context window), long-term (vector DB), episodic (past interactions), semantic (world knowledge). (4) **Planning mechanism** — how the agent breaks goals into steps. ReAct (Thought-Action-Observation), Plan-and-Execute, Tree-of-Thought. (5) **Execution loop** — the control flow that keeps the agent running: observe state → think → act → observe result → repeat until goal achieved or max iterations reached.

■ Example:

Production agent anatomy:

Brain: Claude Opus / GPT-4o (reasoning engine)

Tools:

- web_search(query) -> search results
- read_document(url) -> document text
- run_python(code) -> execution output
- query_database(sql) -> query results
- send_email(to, subject, body) -> confirmation

Memory:

- Short-term: last 20 messages in context
- Long-term: Qdrant vector DB of past interactions
- Working: current task state dict

Planning: ReAct pattern

Thought: 'I need to check inventory before ordering'

Action: query_database('SELECT stock FROM inventory WHERE product_id=123')

Observation: 'Stock: 5 units'

Thought: 'Stock is low, I should send reorder email'

Action: send_email(...)

Loop: runs until task_complete=True or max_steps=20

■ *Interview Tip: When asked to design an agent, explicitly list all 5 components. Most candidates jump to 'use LangGraph' without explaining the components. Listing Brain + Tools + Memory + Planning + Loop shows architectural thinking.*

Q3

What is the ReAct pattern and why is it the foundation of modern AI agents?

Answer:

ReAct (Reasoning + Acting) is an agent prompting pattern where the LLM interleaves **Thought** (reasoning about what to do), **Action** (calling a tool), and **Observation** (reading the result). This loop continues until the agent reaches the final answer. Why it's foundational: (1) It makes reasoning **explicit and traceable** — you can see WHY the agent took each action. (2) It allows **self-correction** — if an action fails (Observation shows error), the agent can think differently and try again. (3) It prevents

hallucination — agent must ground decisions in actual tool outputs, not imagined results. (4) It's **composable** — works with any tool, any LLM, any task. Most agent frameworks (LangGraph, LangChain, CrewAI) implement ReAct at their core.

■ **Example:**

ReAct agent solving: 'What is the population of Tokyo and is it bigger than New York?'

Thought: I need to find Tokyo's population first.

Action: web_search('Tokyo population 2026')

Observation: 'Tokyo population: approximately 13.96 million (city proper)'

Thought: Now I need New York's population for comparison.

Action: web_search('New York City population 2026')

Observation: 'New York City population: approximately 8.34 million'

Thought: I have both numbers. Tokyo (13.96M) > New York (8.34M). I can answer.

Final Answer: Tokyo has a population of ~13.96 million, significantly larger than New York City's ~8.34 million. Tokyo is approximately 1.67x larger.

Without ReAct: LLM might hallucinate population numbers from training data.

With ReAct: Every fact is grounded in actual web search results.

■ *Interview Tip: Contrast ReAct with direct LLM response. 'Without ReAct, the LLM would hallucinate statistics from training data. With ReAct, every claim is grounded in an actual tool result.' This shows you understand WHY ReAct matters, not just WHAT it is.*

Agentic RAG

Q4

What is Agentic RAG and how is it fundamentally different from standard RAG?

Answer:

In **standard RAG**, retrieval is a fixed, one-shot pipeline: embed query → search vector DB → retrieve top-K chunks → generate answer. No flexibility, no iteration, no intelligence in the retrieval process. In **Agentic RAG**, an AI agent **controls and adapts the entire retrieval process**: (1) **Decides WHEN to retrieve** — maybe the agent already knows the answer from memory. (2) **Decides WHAT query to use** — reformulates queries, tries multiple search strategies. (3) **Evaluates retrieval quality** — checks if retrieved chunks actually answer the question. (4) **Iterates if needed** — re-queries with different terms if first retrieval fails. (5) **Routes to different indexes** — billing questions go to billing KB, technical questions go to technical docs. (6) **Combines tools** — RAG + SQL + web search + API calls in one

workflow.

■ Example:

Standard RAG:

User: 'What is our Q3 revenue compared to last year?'

Pipeline: embed -> search docs -> retrieve chunks -> generate answer

Problem: Docs have Q3 this year but not last year. Answer is incomplete.

Agentic RAG:

Agent thinks: 'This needs both current and historical data'

Step 1: Search internal docs for 'Q3 revenue current year'

Step 2: Evaluates: 'Found Q3 2026: \$4.2M. Need 2025 data.'

Step 3: Queries financial database for 'Q3 2025 revenue'

Step 4: Evaluates: 'Got Q3 2025: \$3.1M from database.'

Step 5: Calculates: '\$4.2M / \$3.1M = 35.5% growth'

Step 6: Generates comprehensive answer with both years + growth rate

Standard RAG would give incomplete answer.

Agentic RAG dynamically combined docs + database to get complete answer.

■ *Interview Tip: The key word is 'adaptive retrieval.' Say 'In Agentic RAG, the agent adapts its retrieval strategy based on what it finds. Standard RAG is a fixed pipeline. Agentic RAG is an intelligent decision loop.' This is the core distinction interviewers want.*

Q5

How do you implement query routing in Agentic RAG?

Answer:

Query routing in Agentic RAG means intelligently directing queries to the most appropriate data source or retrieval strategy. Types of routing: (1) **Index routing** — different knowledge bases for different domains (billing KB, technical KB, HR KB). A classifier or LLM decides which index to search. (2) **Tool routing** — some queries need vector search, others need SQL, others need web search. Agent decides which tool. (3) **Strategy routing** — simple queries → dense search only. Complex queries → hybrid search + reranking. Multi-hop queries → decompose into sub-queries. (4) **Model routing** — simple retrieval → cheap model. Complex reasoning over retrieved content → expensive model. Implementation: use an LLM classifier as the router, or a smaller dedicated classification model for speed.

■ Example: